THOMAS MORE GEEL • TOEGEPASTE INFORMATICA • DEEP LEARNING & COMPUTER VISION

Computer Vision Challenge: Sports Vision & Tactical Mapping

Automatische objectdetectie, 2D-homografie en tactische match-analyse van PSG vs. Inter

Team 3: Git Push & Pray

Dmitrij Tchervonisjenko **Kobe Vandenberk** **Cisse Vandeweyer** **Brent Van Gelder**

Video: PSG vs. Inter (660 frames, 30 FPS) • Model: YOLO26l (26.2M) • PyTorch, CUDA & OpenCV • Deployment: Streamlit



SPELERS F1-SCORE	ROLACCURAATHEID	HOMOGRAFIEFOUT	BAL TERUGVINDING
0.99	**99.2%**	**0.16 m**	**19 / 21**
Gemeten op 28 GT-frames	321 van 324 rollen correct	Mediane fout via RANSAC	Op 1280px testresolutie

Projectomschrijving & Doelstelling

Voor deze Computer Vision Challenge ontwikkelen we een toepassing die voetbalbeelden automatisch analyseert en omzet naar een overzichtelijke tweedimensionale tactische weergave van het voetbalveld. De toepassing vertrekt vanuit een vooraf opgenomen voetbalvideo van 22 seconden (660 frames, 30 beelden per seconde, 1920 × 1080) van de topaffiche tussen Paris Saint-Germain en Inter Milan.

Het model herkent automatisch alle belangrijke objecten op het veld: de spelers van beide teams, de doelman, de scheidsrechters en de voetbal. De gedetecteerde objecten worden rechtstreeks op de oorspronkelijke videobeelden aangeduid met behulp van bounding boxes en bijhorende labels. Naast de geannoteerde videobeelden voorzien we een tweede weergave in de vorm van een 2D-bovenaanzicht van het veld waarop de posities synchroon worden geprojecteerd.

Het hoofddoel van dit project is om aan te tonen hoe computer vision ruwe uitzendbeelden kan transformeren naar bruikbare ruimtelijke informatie. Hierdoor krijgen trainers en analisten direct inzicht in formaties, onderlinge afstanden, dekkingsgraden en balbezit.

1. Aanpak, Obstakels & Oplossingen

Targeted Data Annotation & Labelprotocol

De eerste stap was het samenstellen van een kwalitatieve dataset. We extraheerden alle 660 frames uit de video en brachten deze onder in Roboflow. In totaal hebben we 136 representatieve frames gelabeld, goed voor **3.305 bounding boxes** verdeeld over vijf klassen: `ball` (105 stuks), `player_A` (PSG donkerblauw, 1.351 stuks), `player_B` (Inter geel, 1.348 stuks), `player_B_Goalkeeper` (doelman oranje, 134 stuks) en `referee` (turquoise/groen, 367 stuks).

Om consistent te labelen hebben we vooraf duidelijke afspraken vastgelegd in een **annotatieprotocol**:

- **Geen grondschaduw:** De bounding box sluit strak aan op het lichaam en de bal, zonder schaduwen mee te nemen.
- **Motion blur meenemen:** Bij snelle acties werd de volledige bewegingsveeg van de bal in de box opgenomen om het netwerk te leren dat een onscherpe veeg ook de bal representeert.
- **Occlusie en beeldrand:** Een bal werd enkel gelabeld als minstens 30% zichtbaar was; aan de rand van het beeld moest meer dan de helft zichtbaar zijn.
- **Peer review:** Elk teamlid controleerde het labelwerk van een ander teamlid om inconsistenties meteen te corrigeren.

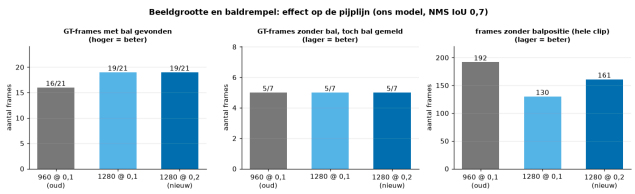
Datasetvoorbereiding en Preventie van Data Leakage

Omdat opeenvolgende videoframes sterk op elkaar lijken, zou een willekeurige splitsing leiden tot **data leakage**: het netwerk herkent dan naburige beelden uit het geheugen in plaats van generaliseerbare patronen te leren. We hebben daarom gekozen voor een **temporele opsplitsing in blokken** met bufferzones tussen train, validatie en test. Daarnaast werden statische duplicaatframes (veroorzaakt door de conversie van 25 naar 30 FPS) verwijderd.

Model Fine-Tuning & GPU-Acceleratie

Tijdens de eerste trainingen duurde het proces op de CPU te lang. We hebben PyTorch geconfigureerd met **CUDA op dedicated NVIDIA GeForce RTX GPU's**, waardoor de training automatisch versnelde en een volledige run van 100 epochs binnen circa 25 minuten afgerond was. Hierdoor konden we negen trainingsruns vergelijken.

Het finale model is **YOLO26l** (26,2 miljoen parameters), getraind vanaf COCO-basisgewichten over 100 epochs (batch size 2, gradient accumulation tot 64, AdamW met cosinus learning rate decay en early stopping na 25 epochs). Belangrijke keuzes bij de data-augmentatie: **Mosaic uitgeschakeld** (zodat de kleine bal niet kromp tot minder dan 8 pixels), lichte Mixup (0.15) en scaling voor robuustheid tegen overlappende spelers, en beperkte HSV-kleuraugmentatie (tintvariatie slechts 0.015) om de shirtkleuren betrouwbaar te houden.



Figuur 2: Effect van inferentiebeeldgrootte op baldetectie: 1280px vindt 19 van de 21 ballen terug (tegenover 16 op 960px).

Vastgestelde Foutanalyse & Gerichte Oplossingen

Belangrijkste Foutbronnen en Oplossingen

- **Spelersoverlappingen:** Bij drukte rond de middellijn en het doelgebied zag het model soms één gecombineerde bounding box. Dit losten we op door Non-Maximum Suppression (NMS) strak af te stellen en een dubbelenfilter (IoU > 0.6) toe te passen.
- **Baldetectie & Valse Positieven:** De bal is klein (ongeveer 16 pixels) en werd soms verward met witte schoenen of de penaltystip. We hebben dit opgelost door de inferentie uit te voeren op **1280 pixels** in plaats van 960 (waardoor 19 van de 21 ballen werden gevonden, zie Figuur 2), de confidence-drempel voor de bal op 0.20 te zetten en een ruimtelijke uitsluiting toe te passen rond de penaltystippen.

Homography & 2D Minimap Projection

Om de spelers en de bal vanuit het schuine perspectief van de tv-camera nauwkeurig om te rekenen naar een tweedimensionaal bovenaanzicht van 105 × 68 meter, maken we gebruik van een **homografietransformatie** via OpenCV (`cv2.findHomography`). Hierbij geldt de **planar assumption**: we beschouwen het speelveld als een plat tweedimensionaal vlak (Z = 0).

Een veelvoorkomend probleem bij homografie is perspectivische parallaxfout: wanneer het middelpunt van een spelerbox wordt getransformeerd, belandt de positie door de hoogte van het lichaam meters te ver naar achteren (een spelerhoofd belandt gemiddeld 6,8 meter te ver). We lossen dit structureel op door niet het boxmidden te projecteren, maar het exacte contactpunt met het gras bij de **voeten van de speler** (`x_center, y_bottom`).

Voor de continu meebewegende en zoomende uitzendcamera hebben we **305 veldpunten gelabeld over 39 keyframes** (hoeken van het strafschopgebied, doelgebied, doellijn, penaltystippen en middellijn). Tussenliggende frames worden temporeel geïnterpoleerd. Door uitschieters en foute kliks automatisch weg te filteren via het **RANSAC-algoritme**, behalen we een mediane herprojectiefout van slechts **0,16 meter** op de geteste veldpunten.

2. Lokale Interface & Deployment (Streamlit Dashboard)

Voor de eindgebruiker bouwden we een performante, interactieve webapplicatie met behulp van **Streamlit** (op te starten via `streamlit run app/app.py`). De interface biedt een intuïtief dashboard waarin de gebruiker de videoclip kan laden, drempelwaarden kan aanpassen en de analyse realtime kan volgen.

De interface bestaat uit twee gesynchroniseerde panelen: enerzijds de originele videoweergave met bounding boxes, teamidentificatie, scheidsrechters en de bal, en anderzijds een meelopende 2D-tactische minimap waarop alle spelersposities en camerabegrenzingen (Field of View) synchroon worden geprojecteerd.

> **Detectie-Caching & Kwaliteitsborging**
>
> Om de applicatie responsief te houden op apparaten zonder dedicated GPU, worden de YOLO-detecties eenmalig berekend op een lage confidence-drempel (conf 0.05) en opgeslagen in een cachebestand op basis van een unieke MD5-inhoudshash van de videoclip en de modelgewichten. Hierdoor kan de gebruiker in de Streamlit-sidebar live spelen met drempelwaarden zonder wachttijd.
>
> De codebasis is strikt modulair gestructureerd (scheiding van detectie, homografie, tracking en visualisatie) en wordt bewaakt door **77 automatische unittests** via pytest, waarmee wiskundige transformaties, matrixinterpolaties en gegevensformaten continu gevalideerd worden.

3. The Extra Mile: Technische Uitbreidingen (Extra Credits)

Bovenop de basisvereisten van de challenge heeft ons team vier geavanceerde technische extensies geïmplementeerd om de robuustheid, consistentie en analytische waarde van het platform naar een professioneel niveau te tillen:

Tracking en Trajectorie van de Bal (ByteTrack & Kalman-filter)

Spelers worden over opeenvolgende frames stabiel gevolgd met **ByteTrack** via het Hongaarse algoritme op basis van bounding box overlapping en bewegingsvoorspelling. Elke speler behoudt hierdoor een uniek ID doorheen de clip. De uiteindelijke teamtoewijzing wordt bepaald door een confidence-gewogen meerderheidsstemming over alle gedetecteerde frames van dat track-ID, wat resulteert in een **rolconsistentie van 99.04%** (geen flikkerende teamkleuren).

Voor de bal ontwikkelden we een op maat gemaakt **Kalman-filter in veldmeters** (toestand vector: x, y, vx, vy) met een dynamische zoekpoort (2 m basispoort + 1.5 m per frame zonder meting). Dit filter overbrugt occlusies en gemiste detecties tot 15 frames (0.5 s) via lineaire toestandsextrapolatie en negeert statische reflecties zoals de penaltystip.

Ruimtelijke Veldbezetting en Warmtekaarten (KDE)

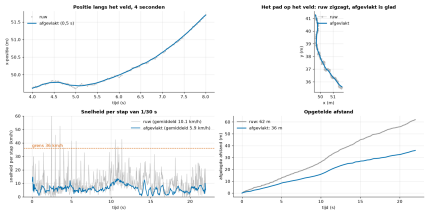
We genereren tactische bezettingskaarten met behulp van **tweedimensionale Gaussiaanse Kernel Density Estimation (KDE)** op de getransformeerde veldposities. Hiermee worden de ruimtelijke dominantiezones en de formaties van PSG en Inter direct zichtbaar gemaakt voor tactische coaches (zie Figuur 3).

Geautomatiseerde Wedstrijdstatistieken & Trajectorie-afvlakking

Het platform berekent continu de afgelegde afstanden, snelheden en balbezit. Omdat lichte trillingen van de tv-camera leidden tot een overschatting van de loopafstanden (zonder correctie bedroeg de cumulatieve meetafwijking +59%), passen we een **afvlakking toe via een voortschrijdend gemiddelde over 0,5 seconden** (zie Figuur 4). Daarnaast analyseert het systeem continu het balbezit met hysterese (PSG 42.3%, Inter 27.1%, Betwist 30.7%) en herkent het schoten op doel.



Figuur 3: Territoriale bezettingsheatmaps (PSG links, Inter rechts) via 2D Gaussiaanse KDE.



Figuur 4: Ruwe vs. afgevlakte trajectorie (elimineert 59% meetruis door cameratrilling).

4. Gebruik van Generatieve AI Tools (GenAI)

Tijdens het ontwerpproces heeft ons team doelgericht en transparant samengewerkt met Generatieve AI-assistenten (Claude Code, Gemini en GitHub Copilot). De AI-tools werden ingezet als sparringpartner en productiviteitsversneller binnen duidelijke kaders:

- **Omgevingsconfiguratie & CUDA-optimalisatie:** Assistentie bij het correct inrichten van PyTorch met CUDA FP16-ondersteuning op NVIDIA RTX GPU's en het beheren van videogeheugen.
- **Modulaire Architectuur & Testautomatisatie:** Ondersteuning bij het ontwerpen van een schaalbare datastructuur en het schrijven van de testsuite van 77 pytest unittests.
- **Wiskundige Verificatie:** Controle van de algebraïsche stappen bij de Direct Linear Transform (DLT), homogene coördinaten en de toestandsovergangen van het Kalman-filter in meters.
- **Kritische Menselijke Evaluatie:** Elk gegenereerd script en voorstel werd handmatig gecontroleerd, door het team doorgelicht en lokaal getest op de dataset voordat het in de repository werd opgenomen.

5. Conclusie & Final Remarks

Met dit project tonen we aan dat een zorgvuldig geannoteerde dataset van 136 beelden en een goed doordachte computer vision-pijplijn volstaan om professionele wedstrijdanalyses uit te voeren. De combinatie van YOLO26l, ByteTrack, homografische veldprojectie en Kalman-filtering levert een betrouwbaar en reproduceerbaar eindresultaat:

Pijplijncomponent	Gebruikte Technologie	Gevalideerd Resultaat & Metriek
Objectdetectie	YOLO26l op 1280px inferentie	F1-score 0.99 op spelers, 19/21 ballen teruggevonden
Spelerstracking	ByteTrack + Majority Voting	41 unieke tracks, 99.04% rol- en teamconsistentie
Bal State Estimation	Kalman Filter in meters + Gating	Occlusie-overbrugging tot 15 frames, stabiel traject
Veldhomografie	RANSAC met 305 punten over 39 keyframes	Mediane projectiefout van 0.16 m op het veld
Tactische Analytics	2D Gaussiaanse KDE + 0.5s filter	Afstandsafwijking -59% gecorrigeerd, live balbezitanalyse
Stabiliteit & CI	Detectiecache + 77 pytest unittests	100% slaging op tests, directe lokale start zonder GPU

De belangrijkste lessen uit dit project zijn de noodzaak van een temporele split ter voorkoming van data leakage, het uitschakelen van mosaic-augmentatie bij zeer kleine objecten, en de grote winst van inferentie op 1280px voor baldetectie. Als mogelijke vervolgstap kan geautomatiseerde keypointdetectie (bijvoorbeeld via YOLO Pose) worden onderzocht om manuele keyframe-selectie volledig te automatiseren.